

Computational Identity and Computational Functionalism

August 2026

What this note does

This note has three deliberately separated parts.

Part I develops the problem landscape through thought experiments. Each case isolates a failure of an overly simple notion of computation and extracts the conceptual distinction that the final theory must preserve: state relabeling, function-versus-algorithm, local abstraction, non-linear granularity, the stone-factorization challenge, participation versus value support, and the split between actual and counterfactual organization.

Part II assembles those constraints into a theory of computational identity. Physical episodes support structured families of admissible computational abstractions, later called computational spectra, grounded in coherent value representation and actual computational provenance; a modal counterpart describes the stronger mechanism across unrealized alternatives. This part is entirely independent of any theory of consciousness.

Part III explores how that computational ontology can be used to formulate computational functionalism more carefully. It does not propose which computational property is sufficient for consciousness. Instead, it develops philosophically motivated naturality considerations: which dependencies look plausible, which would be surprising, what role counterfactual structure might legitimately play, and why one may prefer to distinguish the process actually instantiated from unrealized alternatives.

1 Diagnostic problems and thought experiments

1.1 Why trajectories and decoders are too weak

Thought experiment 1: Why a state sequence is too weak

Setup. Imagine that a computation is represented only by a finite sequence of internal states: first state *A*, then *B*, then *C*, and so on. Now take any physical object whose global microstate changes over time—a stone, a turbulent fluid, a processor, a brain—and record a sequence of distinct snapshots.

First failure: retrospective relabeling. A dictionary can be invented after the fact: call the first physical snapshot “computational state *A*,” the second “state *B*,” the third “state *C*,” and so forth. If implementation requires only correspondence between two state sequences, almost any sufficiently rich finite history can be made to implement almost any finite run. The alleged computation does no explanatory work; the dictionary does all of it.

A tempting repair: require one fixed decoder. Suppose we now demand a single stationary local decoding rule throughout the episode. This blocks the crudest case in which the same physical condition is assigned different meanings at different times.

Why the repair is still insufficient. If every exact carrier microstate encountered during the finite run is unique, a single stationary lookup rule can still be defined after the fact, assigning each encountered carrier state whatever abstract value the target trajectory requires. The decoder no longer mentions time, yet the completed answer remains hidden in its graph. Stationarity prevents contradictory interpretation of repeated carrier states; it does not prevent arbitrary labeling of states seen only once.

What this forces us to add. A realized computation cannot be a bare sequence of atomic

global states, and coherence of the decoder is necessary but not sufficient. The representation must preserve internal organization of the actual process: which events occurred, which value-bearing tokens were present, how tokens were routed, which events consumed them, what new tokens were produced, and which parts of the process were independent. In addition, the eligible carrier variables and decoder family must be fixed independently of the desired computation. Useful admissibility constraints can include physically natural coarse variables, locality, bounded descriptive complexity, and stability under an explicitly stated class of perturbations or across independently selected episodes.

The value interpretation therefore need not be one unique physical code for the entire system. It may be a stable family of local or typed decoding rules, with explicit context-dependent translations where needed. What is excluded is an ad hoc interpretation whose only justification is that it makes the completed trajectory fit the target computation.

Concept forced by the example

The first primitive object should be an **actual valued computational provenance structure**, not a state sequence. Its values must be read through a **coherent representation scheme**, while the eligible carriers and decoder family are constrained by an admissibility policy fixed independently of the target trajectory. A fixed decoder alone does not solve trivial relabeling.

1.2 Function, algorithm, and local abstraction

Thought experiment 2: The same function, two different computations

Setup. Consider the function that maps an integer N to the sum of the integers from 1 to N . One routine initializes an accumulator and adds 1, then 2, then 3, and so on. Another evaluates the closed-form expression $N(N + 1)/2$.

Why input–output equality is insufficient. Both routines compute the same input-output function. Yet they plainly do not instantiate the same internal computation. They generate different intermediate values, different dependency graphs, different timing behavior, different possibilities for interruption, and different suboperations.

So “same function” is too coarse to mean “same computation.”

Required distinction. Distinguish the **extensional function** from the **algorithmic or causal organization** that realizes it. Function identity is one possible abstraction of computation, usually a very coarse one.

Concept forced by the example

A function forgets the path. A computation, at a finer description, includes the structured production of the result. Therefore function identity and computational identity must not be primitive synonyms.

Thought experiment 3: The subroutine disappears inside a larger program

Setup. Put those two summation routines inside two larger programs that are otherwise identical. Suppose the surrounding code only supplies N and receives the returned sum. It cannot inspect intermediate accumulator values, pause the routine halfway through, or observe its internal memory.

At a fine description, the two programs differ because one contains a loop and the other a closed-

form computation. At the interface seen by the surrounding program, the two subroutines are interchangeable.

Why this is not a contradiction. We now have two apparently conflicting statements:

- the programs contain different internal computations;
- the programs can be treated as the same larger computation once the summation subcomputation is viewed only through its interface.

Neither statement should be discarded.

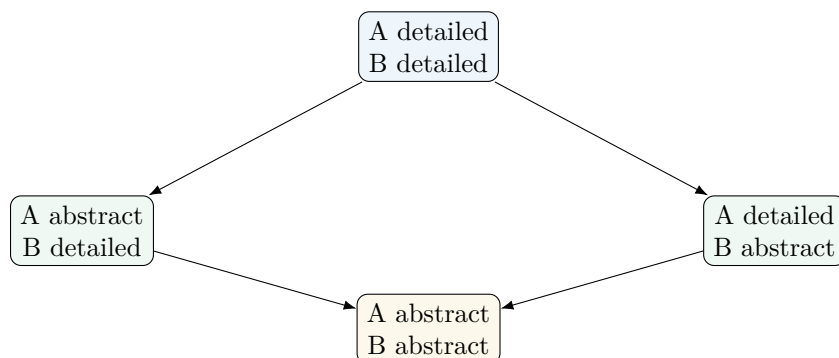
Required idea. Computational identity is always identity *at an abstraction*. A valid abstraction may collapse a whole subcomputation into a single abstract operation when the distinctions inside that subcomputation make no difference to the surrounding structure being retained.

Concept forced by the example

Abstraction is not denial of the finer computation. It is a quotient of it: several distinct fine structures may occupy the same abstract computational role.

Thought experiment 4: Granularity is not a slider

Setup. Now imagine a program containing two complicated routines, *A* and *B*. We can retain the internals of both, hide the internals of *A* while retaining *B*, hide *B* while retaining *A*, or hide both.



Where the scalar picture breaks. The two middle descriptions are incomparable. Neither is simply “more detailed” than the other. One keeps information about *A* that the other discards; the other keeps information about *B* that the first discards.

Therefore a global scalar called “the level of granularity” is misleading.

Required structure. Computational abstractions form a **partially ordered family** under refinement. In sufficiently regular cases this family can have lattice-like structure: common refinements, common coarse-grainings, and many incomparable branches.

Concept forced by the example

Granularity is closer to a structured vector than to a scalar. More precisely, **abstraction is structured information loss**: what matters is not how much detail is discarded, but which distinctions are discarded.

Terminology introduced here: computational spectrum

The **computational spectrum** of a realization is the structured family of its admissible computational abstractions, ordered by refinement. The point of the term is precisely to avoid speaking as if there were one privileged scalar “level.” Different branches of the spectrum can retain different local details and can be incomparable. Part II will state this notion more systematically.

1.3 What an actual execution must contain**Thought experiment 5: The stone that allegedly factors a number**

Setup. Hand someone a huge semiprime and a stone. Suppose they claim that, with a sufficiently ingenious interpretation of the stone’s changing microstate, the stone is performing the factorization.

The challenge is simple: identify where the input is represented, where the intermediate factors or arithmetic quantities are represented, how later computational events actually depend on earlier value carriers, and where the output can be read without placing the factorization itself inside the decoding dictionary.

Where the alleged computation can hide. A global state-labeling map can always hide arbitrary work in the interpretation. But that does not show that the relevant arithmetic organization occurred inside the physical process.

What must be witnessed. The actual-run notion must preserve **locality, provenance, value flow, and causal decomposition**. It must also use a coherent representation scheme for values. An admissible computational interpretation cannot obtain its structure solely by giving each whole-system snapshot an arbitrary name or by changing the decoding rule opportunistically from one event to the next.

What not to conclude

This does not prove that a stone realizes no computations. A sufficiently rich physical episode may genuinely contain computational patterns. The claim is narrower: if a computation is present, it must be witnessed by structure in the episode itself rather than by retrospective global relabeling.

Thought experiment 6: Participation, contribution, and value support

Setup. Consider a three-input OR operation. In one actual occurrence it receives

`true, true, false → true.`

All three input signals physically arrive at the operation event. Should the occurrent computational structure contain all three inputs? Yes, if the relation means that a value token was actually supplied as an operand. But that is not yet the same as saying that each token was important to the particular output value.

There is a useful stronger notion. Relative to the operation OR and to this actual input tuple, a **value support** for the observed output is a subset of the actual input-value assignments that is sufficient to fix that output while the remaining inputs are left unspecified. Here either `input 1 = true` alone or `input 2 = true` alone suffices to guarantee the output `true`; `input 3 = false` does not. The two true inputs therefore form two alternative minimal supports.

For an AND gate receiving `true`, `true`, by contrast, the observed output `true` has one joint minimal support containing both true inputs. For an XOR gate, fixing the parity result generally requires a different support pattern again.

Why one causal edge is not enough. A single edge relation cannot carry all of this information. At least four questions have been mixed together:

- Which tokens actually arrived and participated in the event?
- Which actual-value subsets are sufficient to support the particular output value?
- Which particular values count as causes or contributors under some theory of actual causation?
- Which input positions can affect the operation across alternative cases?

The first question is directly occurrent. The fourth is fully modal. The second is anchored to an actual occurrence but uses the local operation semantics to quantify over alternatives for the unspecified inputs; it is therefore **occurrence-indexed but locally modal**. The third is a further explanatory notion and is delicate under redundancy and overdetermination.

Required distinction. The base occurrent object should record **provenance and participation**. It may be enriched by a **value-support hypergraph**: for each actually produced value, record its minimal sufficient sets of actual value tokens relative to the local operation semantics. Actual contribution and full modal dependence should remain separate notions.

Concept forced by the example

Keep four structures distinct:

Participation / provenance: this token was actually transported to, consumed by, or produced by this event.

Value support: this set of actual value assignments is sufficient to fix this actual result, relative to the local operation semantics.

Actual contribution: this actual value counts as a cause or contributor to this result under an explicit contribution criterion.

Modal dependence: this variable or port can affect the operation across admissible alternative cases.

What not to conclude

A value-support structure is not purely non-counterfactual. It is indexed by an actual occurrence, but sufficiency is assessed using the local rule of the operation. This makes support useful for analyzing how a result is organized without pretending that it belongs to the same ontological layer as raw provenance.

Thought experiment 7: A result produced by silence or inhibition

Setup. A downstream event occurs only if no veto signal arrives during a fixed interval. In another system, a low-voltage token explicitly carrying the Boolean value `false` arrives at a port. Both cases may be described informally as “false,” but their physical provenance differs: one contains a value-bearing token, while the other depends on a non-occurrence over time.

Where positive-event provenance is incomplete. A graph containing only positive event and token occurrences cannot represent absence merely by omitting a node: omission is indistinguishable from an incomplete observation or an arbitrarily cropped episode. Reifying every silence as a token also begs the question of which interval and which missing event make that token legitimate.

Required extension. The base formalism is adequate for positive value flow but not universal. Applications involving inhibition, prevention, timeouts, or alternative enabling need explicit temporal windows and absence/enabling structure, or a translation into such structure whose admissibility is independently justified.

1.4 Actual occurrence versus modal mechanism

Thought experiment 8: One actual run does not identify the whole mechanism

Setup. Observe three actual operation occurrences, all classified under the same local operation schema. On the three input cases that occur, their outputs agree with an AND rule. The fourth input case never occurs.

Or observe a brain for one minute in perfect detail. Suppose we know every actual computational dependency that occurred during that minute, but the subject never encountered millions of other possible sensory inputs.

What the episode leaves open. From the actual episode alone, we cannot infer every unrealized transition. Many distinct mechanisms may agree perfectly on everything that occurred and diverge only on cases that did not occur.

If we insist that every statement about computation must already include all these unrealized cases, then the computational identity of an actual episode becomes immediately counterfactual. If we refuse all counterfactual structure, we should not pretend that the actual run uniquely determines the full mechanism.

Required split. Split the theory into two legitimate notions.

Concept forced by the example

Occurrent computation asks what computational organization was actually instantiated in this episode.

Modal computation asks what computational mechanism the system implements across the relevant alternative inputs and interventions.

The occurrent notion deliberately concludes less. It is stronger than a state trajectory because it contains actual represented provenance and composition, but weaker than a complete modal mechanism because it leaves unvisited cases open.

Thought experiment 9: Same path, different probabilities

Setup. Two stochastic mechanisms produce exactly the same finite event-token episode. Under the first mechanism the observed path had probability 0.9; under the second it had probability 10^{-30} and occurred by extraordinary luck.

Why realized path and stochastic mechanism are different. Path identity alone does not determine whether the transition kernels belong to the identity of the computation. Conversely, kernel identity alone forgets which sample path actually occurred. Treating probability as either automatically occurrent or automatically irrelevant collapses two different questions.

Required split. A stochastic extension needs both a realized occurrence and a modal probability kernel. It must state separately whether probabilities help individuate the represented mechanism and whether a particular comparison is about the actual path, the capacity distribution, or both.

1.5 Comparing realizations and resource tradeoffs

Thought experiment 10: When different computers run the same thing

Setup. Take two old PCs from different manufacturers. Their transistor layouts differ, their low-level timings differ, perhaps parts of their architecture differ, yet both run Windows 95 and behave similarly at familiar software interfaces.

Why binary sameness is the wrong comparison. It would be wrong to say that their entire computational organization is identical. It would also be wrong to say that they share nothing computationally simply because their hardware is different.

Moreover, “Windows 95” itself is not a mathematically complete specification of one abstract computation. It is also the name of a historical software artifact, with versions, binaries, provenance, and conventions about what counts as the same software.

Better comparison. Two systems can realize many of the same computational abstractions while differing at finer or orthogonal abstractions. In the terminology introduced above, their computational spectra can therefore have a large structured overlap without being identical. Ordinary names such as “the same program” may designate some part of that common structure, but the artifact-identity question is not identical to the computational-identity question.

Concept forced by the example

The useful primitive comparison is not merely “same computation: yes or no.” It is: **which computational abstractions are realized by both systems, and how far does their common structure extend before their realizations diverge?**

Thought experiment 11: Hardware reuse and finite unrolling

Setup. Consider two physical realizations of the same finite computational episode. In the first, one physical logic unit is used repeatedly at successive moments. In the second, each successive operation is carried out by a fresh physical copy. Arrange the second realization so that the same operation occurrences take place in the same order, with the same represented values and the same dependencies between produced and consumed tokens.

The tempting but irrelevant distinction. One may be tempted to say that the first episode contains “the same component acting again” whereas the second contains many distinct components. That is a real difference in the physical history of the implementations. But nothing computational follows from it merely as such. If the event structure, represented values, operand/result roles, and computational dependencies are otherwise identical, the fact that two operation occurrences happened to be realized by one recurring piece of hardware rather than two pieces of hardware should not split the computation into different computational types.

Conversely, an unrolling may genuinely change the computation when it changes the actual control structure, available intermediate values, dependencies, timing relations retained by the abstraction, or other computational organization. The point is not that every loop equals every unrolling. The point is that *persistent physical identity is not itself the criterion of difference*.

What this rules out. A theory of computation should individuate computational events by their role in the represented process, not by the continued identity of the physical object that realizes them. Hardware identity belongs to the theory of the implementation substrate unless it leaves a further computationally represented difference in the episode.

Concept forced by the example

Component-identity invariance. Replacing repeated use of one physical device by distinct devices—or conversely reusing one device for several operation occurrences—does not by itself change the computation. What matters is the structure of operation occurrences, represented values, and dependencies, not which persisting chunk of matter realizes each occurrence.

Thought experiment 12: Size constraints and computational overlap

Setup. Model an open synchronous system by a global transition rule: current internal state plus current input determines the next internal state and current output. Now imagine two realizations of exactly the same global rule.

One is a richly factored mechanism with many reusable local computations. The other is an enormous lookup table with one stored answer for every possible state-input pair.

The observation. At the level of the global transition function, the two realizations are identical. At finer computational descriptions they can be radically different: the table spends memory to avoid reproducing most of the factorization exploited by the structured realization. Thus equality of the global function alone does not force much fine computational overlap when resources are effectively unlimited.

Why size changes the question. Now impose a memory or description-size budget. The lookup-table strategy eventually becomes impossible. As the admissible resource budget shrinks, the class of realizations of the same global function also shrinks. It becomes plausible that some internal computational structures must recur across all sufficiently compact realizations, even though there is no general reason to expect the original algorithm to become unique.

What this example suggests rather than defines. This thought experiment does not force a new primitive into the theory. It points toward theorem-shaped questions that become available once computational spectra and their overlap are defined: how does the guaranteed common structure among realizations of one function change as memory, circuit size, description length, time, locality, or other resources are restricted?

A theoretical direction suggested by the example

One can define a **resource-constrained common core**: the computational structure shared by every admissible realization of a given abstract function inside a chosen resource region. Tightening the resource region can only remove candidate realizations, so the inevitable common core can stay the same or grow. Additional assumptions would be needed to prove that the growth is substantial, or that sufficiently tight constraints force anything close to a unique algorithm.

2 The computational theory

The diagnostic cases above are not independent curiosities. Together they constrain a compact computational ontology. The purpose of this section is to assemble that ontology without making any claims about consciousness.

2.1 Physical episodes and computational provenance

A **physical episode** is a bounded piece of the actual evolution of a physical system. For computational purposes, the episode should not be reduced to a sequence of global snapshots. What matters is its internal actual organization: events, typed value-bearing carriers or tokens, operand and result roles, transport, consumption, production, routing, control, declared boundaries, and partial order.

Call this enriched object the episode’s **actual valued computational provenance structure**. The word “provenance” is deliberate. The base object records how pieces of represented information actually moved through and were transformed by the episode. It does not silently identify that factual participation relation with a stronger theory of which values were the decisive causes of each result.

Values require a representation discipline. The theory should not require one unique encoding for the entire physical system: the same abstract Boolean may be electrical in one subsystem, optical in another, and magnetic in a third. What it requires is a **coherent representation scheme**: a stable family of local or typed decoding rules, together with explicit translations where necessary. The scheme must be reusable across occurrences and cannot be selected separately for each point after inspecting the completed history. Context-dependent encodings are allowed when the context and translation rule are themselves part of the represented organization.

This requirement blocks a local version of the relabeling trick. A physical difference counts as a represented computational value only through a representation rule that has some stable domain of application; the decoder cannot smuggle the desired computation into a trajectory-indexed dictionary.

Several actual occurrences classified under a common operation schema can reveal more structure without introducing global counterfactual commitments. Their observed input-output cases may support a **partial operation table**: what instances of that operation schema in fact did on the cases that occurred. Unvisited input tuples remain unspecified at the occurrent level.

A second enrichment is possible when the local operation type is independently available. For one actual operation occurrence, define a **value support** as a set of actual input-value assignments sufficient to fix the actual output value while the remaining inputs are left free. Minimal supports form a hypergraph over the actual value tokens. This object is useful but must be classified carefully: it is anchored to an actual occurrence yet uses local modal semantics to test sufficiency. It is therefore richer than provenance and weaker than importing the modal organization of the whole system.

Working definition

An **occurrent computational abstraction** is a structure-preserving compression of an actual valued computational provenance structure under a coherent representation scheme. It may hide internal details, merge events, or replace a detailed subsystem by an abstract operation, but every retained operand, result, dependency, and precedence relation must have an actual witness. Selecting a subepisode must either retain a selected event’s relevant incident tokens or expose them explicitly at its boundary.

2.2 Four relations that should not be collapsed

The phrase “causal structure” is too overloaded to serve as a single primitive. At least four relations should be kept conceptually separate.

Relation	What it says	Modal status
Participation / provenance	A token was actually delivered to, consumed by, transported through, or produced by an event	Strictly occurrent base relation
Value support	A set of actual input-value assignments is sufficient for the actual result relative to the local operation semantics	Occurrence-indexed, locally modal
Actual contribution	A particular actual value contributed to a particular actual result under an explicit theory of actual causation or contribution	Optional explanatory enrichment
Modal dependence	An input position or variable can affect an output or downstream variable across admissible alternative cases	Fully modal mechanism relation

The distinctions matter even for tiny examples. In $\text{OR}(\text{true}, \text{true}, \text{false}) = \text{true}$, all three tokens participate, the false token belongs to no minimal sufficient support for the true output, and the two true tokens appear as alternative minimal supports. Yet all three ports belong to the gate's modal dependence structure. A theory of actual causation may classify the two redundant true values in still another way.

The support relation is naturally hypergraphical rather than merely graphical. An AND output may require a joint set of values; an OR output may have several alternative singleton supports. Keeping this structure separate prevents the theory from pretending that one binary edge relation captures participation, sufficiency, explanatory contribution, and counterfactual dependence at once.

2.3 Modal mechanisms

A **modal computational mechanism** enriches the actual organization with stable rules across a relevant family of possible inputs and interventions. It says not only how one episode unfolded, but how the same organization would transform other admissible states or respond to perturbations.

Modal structure is what distinguishes an actual partial regularity from a full implemented rule. Three observed instances that are AND-compatible may support an occurrent partial operation; identifying their common operation schema as AND in the full mechanistic sense requires the missing modal commitment.

Working definition

A **modal computational abstraction** is an admissible abstraction of a physical mechanism that preserves the computational organization across the relevant alternatives, not merely along one realized history.

2.4 The computational spectrum

A system should not be assigned one privileged computational level. It supports many admissible abstractions, often incomparable because different subsystems can be abstracted independently.

The structured family of these abstractions is its **computational spectrum**.

For one actual episode we obtain an **occurrent computational spectrum**. For the underlying mechanism we obtain a **modal computational spectrum**.

Schematic notation is useful even in an informal account:

$$\text{Spec}_{\text{occ}}(E), \quad \text{Spec}_{\text{mod}}(M).$$

These are not merely unstructured bags of descriptions. They carry at least a refinement relation: one member refines another when it preserves every distinction retained by the coarser member and adds further structure.

Implementation without an observer-selected level

A physical system implements an abstract computation C when C occurs as an admissible member of the relevant computational spectrum.

The observer may decide to ask about C , but does not create C by selecting a level. More exactly, relative to a carrier, factorization, and decoder policy fixed independently of C , the system either contains the required structure or it does not. Different scientific domains may justify different policies; hiding that parameter would overstate the objectivity claim.

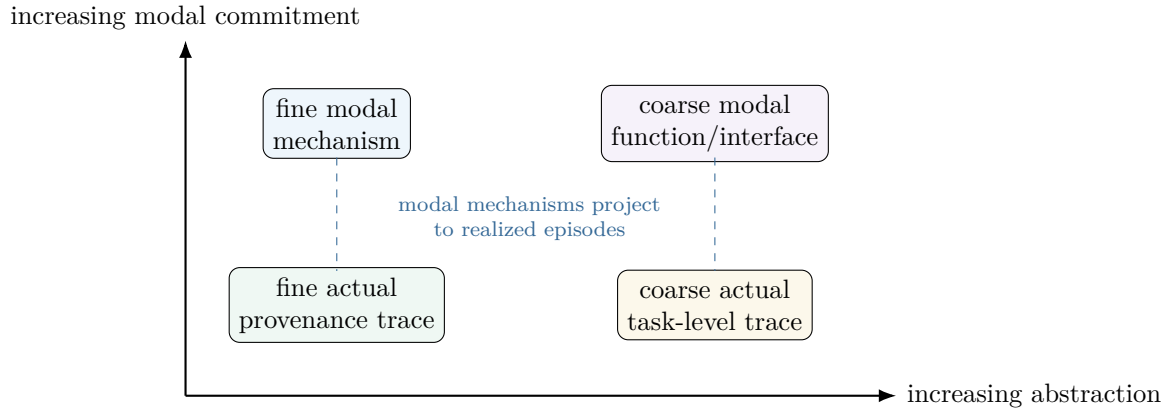
2.5 Why there are two spectra

The occurrent and modal spectra answer different questions.

	Occurrent spectrum	Modal spectrum
Target	One actual episode	The stable mechanism
Contains	Actual events, represented values, participation/provenance, partial operations, local composition; optionally value supports when local operation semantics are supplied	Transition organization and modal dependence across relevant inputs and interventions
Strength	Weaker	Stronger
Leaves open	Unvisited cases	Nothing inside the specified modal domain
Blocks	Mere relabeling of snapshots	Accidental agreement on one history

A modal realization whose variable grouping is backed by local event and port witnesses, when actually executed, should produce a corresponding occurrent realization under the same representation scheme. Global state-transition agreement alone is insufficient, because two mechanisms with the same global transition can have different internal provenance. A single occurrent realization generally admits many possible modal completions. The occurrent trace can constrain those completions by accumulating partial operation tables without selecting values for cases that never occurred.

This relationship can be pictured without implying that abstraction and modality are the same dimension:



2.6 The internal geometry of a computational spectrum

Once the spectrum replaces the picture of one privileged level, several comparative notions become natural.

2.6.1 Refinement

A computation A **refines** a computation B when there is an admissible abstraction from A to B . Before quotienting, this relation is generally only a preorder: two non-identical descriptions may abstract to one another. The spectrum becomes a partial order after identifying mutual-abstraction classes. Replacing a loop by a black-box “SUM” component moves from a finer member of the spectrum to a coarser one. Abstracting a different component gives a different branch, not necessarily a higher or lower point on one scalar ladder.

2.6.2 Containment

A system X has **computational containment** over Y when the relevant computational structure of Y embeds into the spectrum of X . Schematically one may write

$$\text{Spec}(Y) \hookrightarrow \text{Spec}(X),$$

with the understanding that the embedding should respect refinement relations, not merely set membership.

Emulation is a natural source of containment: a richer system may realize the computational organization of another system while also realizing additional structures.

2.6.3 Computational equivalence

Two systems are **spectrally equivalent** in a strong sense when their relevant computational spectra have the same structure. This is much stronger than saying that they compute the same input-output function or run the same software artifact.

Strong spectral equivalence is therefore not the default notion of “same computation.” It is one precise endpoint in a family of comparisons.

2.6.4 Common computational core

The most useful comparison between different systems is often their **common computational core**: the structured family of computational abstractions realized by both.

For systems X and Y , the informal idea is the structured overlap

$$\text{Core}(X, Y) \sim \text{Spec}(X) \cap \text{Spec}(Y).$$

The notation hides technical choices about isomorphism and embedding, but the concept is clear: two machines can be deeply different while still sharing a rich common computational organization.

Every spectrum may also contain trivial bottom abstractions. Their presence in every intersection is not evidence of a substantive common core. Comparisons should therefore identify maximal nontrivial common elements, impose a rank or resource baseline, or explicitly subtract the universal bottom structure.

Takeaway

The question “are these two machines doing the same computation?” is usually underspecified. Better questions are:

- Which computations occur in both spectra?
- Does one spectrum contain a faithful copy of part of the other?
- What is the richest common abstraction?
- At which refinements do the systems diverge?

2.7 Objectivity, context, and naming

The theory separates three issues that are often collapsed.

First: physical realization. Once a realization policy has been fixed independently of the target computation, whether an admissible structure occurs in the spectrum is intended to be an objective fact about the system. Admissibility includes coherent representation, boundary fullness, and provenance witnessing: a computational value or dependency cannot be rescued by changing the codebook opportunistically or by drawing an abstract edge with no physical witness.

Second: abstraction. The spectrum contains many abstractions at once. No single member has to be privileged as “the” computational level.

Third: reference. Human language may be vague about which abstract object is intended. “Windows 95,” “the same algorithm,” or “the same agent” can underdetermine what part of the spectrum is being named.

This locates some conventionality in the question rather than in the physical realization. A name may fail to specify its target precisely; that does not imply that membership in a policy-indexed spectrum is observer-relative. The stronger claim that one admissibility policy is uniquely privileged across all scientific domains is neither needed nor established here.

Context also matters locally. Two subroutines can be interchangeable for a caller that sees only input and output, yet distinguishable for a caller that observes timing, intermediate memory, power use, or interruptibility. The admissible abstraction must therefore preserve every distinction that remains accessible through the interface of the larger computation.

2.8 Consequences that fall out of the model

The model is informal here, but several theorem-like consequences follow almost immediately from the architecture.

1. Bare trajectory identity is too weak

If computational identity preserves only temporal succession of atomic global states, trivial re-labelings are unavoidable. Any nontrivial occurrent notion must preserve more internal structure than a state sequence.

2. Value identity requires a coherent representation scheme

An occurrent implementation cannot be certified by a decoder that changes opportunistically along the trajectory. Representation may be heterogeneous across physical subsystems, but each local or typed code must be stable enough to be reused across occurrences without consulting the completed run.

3. Participation, support, contribution, and modal dependence are different

A value can be an actual operand without belonging to any minimal support of the observed result; a minimal support can be one among several redundant alternatives; an actual-causation criterion can classify those alternatives differently; and a port can belong to the modal dependence structure even when its current token is support-irrelevant. These relations should not be collapsed into one causal edge.

4. Value support is intermediate rather than purely occurrent

Minimal sufficient supports are indexed by an actual event and actual values, but their sufficiency is evaluated using the local operation semantics over unfilled inputs. They are therefore a principled intermediate object between raw provenance and full-system modal structure.

5. Function identity is a quotient, not full identity

Distinct computations can collapse to the same input-output function. Therefore equality of functions can be recovered as a coarse spectral equivalence without being mistaken for equality of finer algorithms.

6. Abstraction is generally only partially ordered

Independent local abstractions generate incomparable descriptions. Existence of an abstraction first gives a preorder; quotienting descriptions that abstract to one another gives the corresponding partial order. A theory that represents granularity by one scalar level discards real structure in the space of computational descriptions.

7. Modal realization projects to occurrent realization

Whenever a modal realization is backed by compatible local event and port witnesses and is actually run, the realized episode instantiates a compatible occurrent member. Commutation of global states alone does not establish this projection.

8. Occurrent realization underdetermines modal realization

Even a complete actual provenance structure need not determine what would have happened in unvisited cases. One occurrent spectrum can therefore be compatible with many modal completions.

9. Local substitution is controlled by interface

If a context can observe only an abstract interface of a subsystem, implementations that are equivalent at that interface may be substituted without changing the coarser surrounding computation. If the context observes additional distinctions, the substitution may cease to be valid.

10. “Same computation” is naturally many-valued in structure, not vague in truth

Two systems may share one computation, many computations, or an entire common subspectrum while differing elsewhere. The richness of shared structure replaces a single primitive yes/no relation.

11. Occurrent repetition can reveal partial laws without becoming modal

If several actual instances of a common operation schema are encountered on different input cases, the occurrent structure can accumulate a partial operation table. This can strongly constrain compatible modal mechanisms while leaving unvisited cases genuinely open. More observation can therefore enrich occurrent computation without erasing the occurrent/modal distinction.

12. Resource restriction can enlarge the inevitable common core

Fix an abstract computation and consider all of its admissible realizations satisfying a resource budget. Intersect their spectra. If the budget is tightened, the class of admissible realizations shrinks, so this inevitable common core can only stay the same or grow.

This does *not* imply that stronger compression always recovers the architecture of some chosen original system. Different compact algorithms may still exist. It instead defines a clean notion of **computational rigidity**: which structures become unavoidable under resource pressure.

Trivial bottom abstractions should be removed or discounted when reporting such a core; otherwise every intersection can look nonempty for an uninformative reason.

2.9 A compact reconstruction of the theory

The formal theory can be reconstructed from this note in the following order.

1. **Represent actual processes richly.** Use events, typed value-bearing tokens, operand and result roles, declared boundaries, actual transport, consumption, production, ordering, control, and composition. Do not begin from atomic state sequences.
2. **Fix coherent value representation and admissibility.** Allow heterogeneous local encodings, but require stable typed decoding rules and explicit translations. Fix the eligible carriers and decoder family independently of the desired trajectory; stationarity alone is insufficient when every exact carrier state is unique.
3. **Separate dependence relations.** Use participation/provenance as the base occurrent relation. Distinguish it from actual contribution and from general modal dependence.
4. **Add value support only as an explicit enrichment.** When local operation semantics are supplied, actual values can be organized into minimal sufficient support sets. Record that this is occurrence-indexed but locally modal, not primitive provenance.
5. **Define admissible abstractions.** An abstraction may forget internal distinctions only when every retained incidence, dependency, and boundary relation has a physical witness.
6. **Order abstractions by refinement.** Allow different subsystems to be abstracted independently, first obtaining a preorder and then its poset reflection; lattice-like structure requires

further conditions.

7. **Build the occurrent spectrum.** For a physical episode, collect all admissible abstractions of its actual valued computational provenance structure. Repeated actual cases may determine partial operation tables.
8. **Build the modal spectrum.** For a mechanism, collect all admissible abstractions whose transition organization and dependence structure remain valid across the relevant alternative inputs and interventions.
9. **Relate the two locally.** Unfolding-compatible modal structures project to actual runs; actual runs and their partial laws constrain but generally do not uniquely determine modal completions.
10. **Define implementation by membership.** A computation is implemented when it occurs in the relevant spectrum, not because an observer has privileged a level.
11. **Compare systems spectrally.** Use refinement, containment, spectral equivalence, and common computational core rather than one undifferentiated relation of sameness.
12. **Keep naming separate.** Artifact labels such as program names may require additional historical or conventional criteria beyond computational structure.
13. **Optionally add resource constraints.** Study how the common core of all realizations changes as memory, description length, time, circuit size, locality, or other resources are restricted.

The final picture

The theory has two primary axes and one optional third axis.

Abstraction axis: which distinctions in the computation are retained? This produces a branching refinement structure rather than one scalar level.

Modal axis: are we describing only represented value flow, participation, and provenance actually instantiated; enriching that occurrence with local support information; or describing the full transition and dependence mechanism across unrealized alternatives?

Resource axis: among all realizations of the same abstract computation, which internal structures become unavoidable when the admissible resource region contracts?

The resulting object is not “the computation of the system” but a structured computational spectrum. Objective implementation becomes spectrum membership; similarity between systems becomes structured overlap; coherent representation and provenance block arbitrary trajectory relabeling at the occurrent level; and the occurrent/modal distinction separates what actually happened from the stronger mechanism across unrealized alternatives.

3 Computational functionalism: naturality constraints suggested by the framework

The preceding sections develop an ontology of computational realization in terms of actual episodes, modal mechanisms, and structured spectra of abstraction. That ontology can then be used to ask a different kind of question:

What properties would it be natural for a consciousness-making computational criterion to have?

Nothing in this part is a theorem of the computational theory, and the discussion does not identify which computational property, if any, is sufficient for consciousness. The aim is more modest: to identify plausible *naturality constraints* suggested by thought experiments. Each proposal below is motivated by an intuition about what would be strange for a theory of a particular conscious

episode to depend on. A substantive theory could reject one of these intuitions, but it would then owe an explicit reason for doing so.

3.1 Why an episode is a natural candidate bearer

An abstract algorithm is a type. It can be instantiated zero, one, or many times. A conscious experience, by contrast, is naturally spoken of as an occurrence: something happening in a particular system over a particular interval.

This makes it natural to investigate theories that assign phenomenal properties primarily to **actually instantiated computational episodes**, or to equivalence classes of such episodes under neutral redescriptions specified independently of the phenomenal verdict. The modal computational spectrum remains important for what the system can do, but one need not assume from the outset that the whole modal spectrum is constitutive of what is experienced in one actual episode.

Phenomenal Actuality: a plausible default

A natural default is that the phenomenal character of an episode depends on properties of the computational process actually instantiated in that episode. A theory may choose to make unrealized modal structure constitutive as well, but such a dependence should be separately motivated rather than silently inherited from a definition of computation.

3.2 Why purely counterfactual differences are philosophically suspect

Suppose every actually encountered instance of a two-input OR gate has input `true,true` and output `true`. Replace those gates by AND gates. On the episode that actually occurs, the same tokens arrive and the same output tokens are generated in the same larger provenance pattern. The two mechanisms differ sharply on `true,false`, but that case never occurs.

It would be philosophically striking if the phenomenology of the completed episode changed solely because the replacement gate *would have* behaved differently on an input that never occurred. Such a theory is coherent, but it makes an actual experience depend constitutively on an unrealized branch. That is a substantive modal commitment, not something that should be obtained for free from the word “functionalism.”

Counterfactual Irrelevance: a plausible default

Once the actual computational structure has been independently grounded, it is reasonable to regard differences confined to unrealized alternatives as phenomenally irrelevant *by default*. A theory can reject this default, but should then explain why those unmanifested alternatives themselves belong to the basis of the actual episode.

This proposal is stronger than merely saying that the system’s observable input-output behavior is the same. The actual internal generative process may still matter. The qualification is essential because modal information can play three different roles:

1. **Individuating:** dispositions may help determine which carriers, operation types, or representations are present in the actual episode.
2. **Evidential:** interventions may reveal which actual dependencies and boundaries the system possesses.
3. **Constitutive:** unmanifested alternatives may themselves be proposed as part of the present phenomenal basis.

The proposed default concerns the automatic move to the third role. It does not ban the first two,

and it cannot be used to validate an arbitrary actual-run decoding whose grounding already fails.

OR versus AND: why value support is not automatically phenomenal

For $\text{OR}(\text{true}, \text{true}) = \text{true}$, the actual output has two alternative singleton minimal supports. For $\text{AND}(\text{true}, \text{true}) = \text{true}$, it has one joint support containing both inputs. Thus the value-support hypergraphs differ even though the actual token flow and generated values can be identical.

This makes value support a useful diagnostic object in the theory of computation, but not an automatic phenomenal constituent. The difference in support is obtained from local counterfactual semantics. A candidate consciousness theory may use support structure if it has an independent reason to do so; absent such a reason, it is natural to hesitate before allowing an unrealized local alternative to change the phenomenology of the actual episode.

Dormant error correction: individuation versus constitution

Suppose the same actual bit sequence passes through two devices, but only one belongs to an error-correcting architecture. No error occurs during the episode. The dormant code need not contribute directly to present phenomenology; nevertheless, it may help determine what counts as a bit, an error, a system boundary, or a correction operation. This case separates a modal fact used to individuate the actual computation from a modal fact promoted to a phenomenal constituent.

3.3 Why actual generation may matter beyond snapshot succession

Counterfactual irrelevance does not imply that any replay of the same sequence of states is equivalent. A movie player can display a sequence of recorded brain states while each frame is produced by reading storage rather than by the preceding computational organization. The state sequence can match while the actual provenance of value production does not.

This gives a *prima facie* reason to allow a consciousness theory to depend on the actual generative organization of an episode rather than only on its ordered snapshots or external input-output trace.

Generative-Process Sensitivity: a plausible allowance

It is legitimate for a computational functionalism to be sensitive to how the actual episode is generated: which represented values are produced, transmitted, combined, and routed across actual events. Equality of snapshots or output traces alone appears too weak to capture the distinction between a generated computation and a mere replay.

This is a possible middle position: richer than pure input-output functionalism, but less modal than requiring the entire counterfactual machine to be preserved.

3.4 Why representation, substrate, and inert additions are natural candidates for invariance

Several transformations have a strong claim to be consciousness-neutral *if* they leave whatever actual computational organization the substantive theory regards as relevant unchanged. The motivation is not that this has been proved, but that allowing such changes to alter phenomenology would make the alleged computational criterion depend on features that seem external to its computational content.

Representation Invariance: a natural expectation

There is a strong *prima facie* reason to expect a coherent renaming or recoding of computational values to leave phenomenology unchanged when the represented computational organization is otherwise preserved.

Substrate Invariance: a natural expectation

Within a genuinely computational functionalism, changing the physical material is naturally expected to be irrelevant when the consciousness-relevant actual computational organization is preserved. A theory that rejects this may still be coherent, but its consciousness criterion is then not purely computational in the intended sense.

Inert-Extension Invariance: a natural expectation

It is natural to expect that adding structure causally isolated from the declared bearer throughout the relevant interval should not, by itself, alter the phenomenology of that episode.

The intuition behind the last case is especially clear: otherwise, attaching a physically inert device whose only role is to change what the system *could* have done may alter present experience without changing anything that actually happens in the episode.

3.5 Why redescription should not multiply experience

The computational spectrum contains many abstractions of one episode. A neuronal description, a circuit description, and a task-level description can all be true at once. It would be difficult to motivate counting each as a new consciousness merely because the same process admits several computational witnesses.

Abstraction Consistency: a natural expectation

A plausible theory should specify a family of neutral redescriptions and refinements using computational or physical criteria independent of the phenomenal verdict, and one would naturally expect its consciousness-making property to be invariant under those neutral transformations. Multiple spectral descriptions of one physical realization do not, merely by existing, provide a good reason to multiply phenomenal subjects or episodes.

This does not solve subject individuation when genuinely distinct or overlapping subsystems each instantiate candidate consciousness-bearing structures. That is a separate problem that a complete theory must address.

3.6 Why robustness and present phenomenality may come apart

Consider an extreme replica of one minute of brain activity. Every local operation is replaced by a brittle device engineered only to produce the correct outputs on the inputs it will actually encounter during that minute. The device would fail on nearby unrealized cases, but the actual generative provenance of the target episode is preserved.

There is an intuitive case for distinguishing two questions:

- the replica and the brain can differ radically in **modal robustness**, future capacity, and counterfactual competence;
- this difference need not automatically imply a difference in the **actual phenomenal episode** if the consciousness-relevant actual process is genuinely preserved.

The alternative is to let unused competence alter the phenomenology of an episode in which that competence never manifests. That may be defensible, but it is again a substantive modal commitment.

Capacity–Occurrence Separation: a plausible distinction

It is natural to locate capacities—which conscious episodes a system could support, how robustly it can continue, and how it would respond to other conditions—in the modal computational spectrum, while treating the phenomenology of the episode currently instantiated as primarily a question about its occurrent computational organization.

A stochastic variant sharpens the issue. Two systems may realize the same finite episode even though the episode is typical under one transition kernel and astronomically unlikely under the other. An actual-process theory must say whether those probabilities merely characterize capacity, help individuate the actual process, or enter phenomenology directly; the slogan “same path” does not decide the matter.

3.7 A family of plausible desiderata for computational-functionalist theories

The preceding considerations do not identify the consciousness-making invariant. They instead describe a family of expectations that seem natural for a computational-functionalist candidate. None is compulsory by logical necessity: a theory may violate one, but the violation becomes a substantive feature that should be defended rather than an unnoticed consequence of the formalism.

A candidate property P is therefore especially well motivated, other things equal, when:

1. P is a property of an actually instantiated computational organization, not merely of an abstract program considered without a realization;
2. P is invariant under coherent value recoding and substrate changes that the theory itself regards as computationally irrelevant;
3. P does not change merely because unrealized transition rules or structures causally isolated from the declared bearer are altered, unless an independent argument makes those modal facts constitutive;
4. P can remain sensitive to actual internal generation and provenance, so that a movie playback need not be equivalent to a genuine computation;
5. P is stable across an independently specified family of neutral abstractions and refinements, rather than defining neutrality by presupposing P 's own phenomenal verdict;
6. if P depends on a locally modal object such as value support, the theory explains why that local counterfactual information is constitutive rather than merely diagnostic;
7. P distinguishes phenomenal occurrence from modal capacity rather than silently treating them as the same question;
8. P states its bearer boundary and does not infer multiple subjects merely from multiple descriptions or overlapping quotients;
9. P gives explicit treatment to diachronic and robustness questions such as subdivision, concatenation, timing variation, noise, and approximation;
10. P is operationally constrainable in principle, so that its extraction rule does not encode the phenomenal verdict it is meant to predict.

A plausible default architecture

These considerations motivate, without proving, a deliberately actualist option: **actual-process computational functionalism**. On this view, phenomenal facts are sought among independently grounded invariants of the computation actually generated during an episode, modulo coherent recoding, substrate changes, neutral refinement, and causally isolated extensions. Modal structure can still individuate mechanisms, support evidence, describe capacities, and constrain possible episodes; the distinctive default is merely that a difference confined to unrealized alternatives is not automatically promoted into a phenomenal difference in the episode that occurred.

3.8 Division of theoretical labor in the combined framework

The computational theory and the functionalist discussion play different roles.

The computational theory tells us, relative to an independently fixed realization policy, what structures are realized: provenance, representation, abstractions, partial operations, optional support structures, modal mechanisms, spectra, refinement, containment, and common computational cores.

The functionalist discussion then asks which ways of using those structures appear philosophically natural for an account of conscious episodes. In particular, the preceding thought experiments give reasons to be cautious about four shortcuts:

- identifying consciousness with a bare input-output function;
- identifying it with a relabelable sequence of states;
- making it automatically sensitive to the entire counterfactual machine;
- treating every computational description of one process as a separate phenomenal realization.

The resulting research question is sharper than “which computation is conscious?” It is:

Which invariants of an actually instantiated computational spectrum are phenomenal, and what additional modal information, if any, has an independent reason to be constitutive?

This leaves the substantive theory of consciousness open while specifying a narrower and more explicit target space for such theories.

One-sentence version

Relative to independently fixed realization policies, physical systems support structured spectra of computational abstractions grounded in coherent representation and witness-full actual provenance; modal structure strengthens this into claims about alternative cases, and this distinction in turn motivates an actualist family of computational-functionalist theories in which present phenomenology is sought primarily in invariants of the generated episode unless an independent argument makes unrealized alternatives constitutive.